Training Priors for Task Independent Features with Natural Images

Texture discrimination: learn the transform to ABR space from natural images

mk_rot_mtrx.m

Steps:

1. Select a 16-bit natural image from, for example, **'\CPS Set-9-10-12_16-bit linear'**
2. Convert to double
3. Normalize all three channels by a single constant so that the max of the three channels to 255
4. Apply approximate optics of the human eye (**aply_otf.m**)
5. Convert to LMS cone space (**rgb2lms.m**)
6. Randomly sample 64 x 64 patches (1^o x 1^o) from each image
7. Normalize each patch to have an average mean across the 3 channels of value of 128 (**ptch_norm.m**)
8. Compile a long list of LMS pixel values by applying steps 1-7 to many images
9. Apply PCA to this list to obtain the transform (coefficients) to ABR space (**pca.m**)
10. Save the transformation matrix (**PCA_matrix_3_OTF.mat**)

Texture discrimination: learn cdfs for task-independent features from natural images

cdfs_of_features.m

1. Select a 16-bit natural image from, for example, **'\CPS Set-9-10-12_16-bit linear'**
2. Convert to double
3. Normalize all three channels by a single constant so that the max of the three channels is 255
4. Apply approximate optics of the human eye (**aply_otf.m**)
5. Convert to LMS cone space (**rgb2lms.m**)
6. Randomly sample 64 x 64 patches (1^o x 1^o) from each image
7. Normalize each patch to have an average mean across the 3 channels of value of 128 (**ptch_norm.m**)
8. Transform to ABR space (**rot.m**), save abr values for color cdfs
9. Select the A channel, contrast normalize to 0.25 (**cntrst_norm.m**)
10. Apply local spatial feature kernels (**imgrad.m**, **imgrad2.m**, **cen_sur.m**), save values for getting spatial cdfs
11. Repeat from step 1 until all natural images are processed
12. Compute cdfs (**histcnts.m**), save cdfs and lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)

Training Task-Dependent Features with Texture Images

Texture discrimination: learn bins for task-dependent features from texture images

opt_bins.m

1. Load cdfs and lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)
2. For the given task-independent feature, set the cdf parameters
3. Load 60 Brodatz and 60 Fabric images; convert to double; for Fabric linearize (**adobe_expand.m**)
4. Apply approximate optics of the human eye (**aply_otf.m**)
5. Convert to LMS cone space (**rgb2lms.m**), down sample for given eccentricity/[lev = 1,2,4,8] (**dsmp.m**)
6. Use the cdf parameters to get the initial bin bounds; e.g., 2 bins = 3 bounds (**mk_bins.m**)
7. Find the next bin bound to test (**find_bnd.m**)
8. Test the bin bounds including the new bound (**test_bnds.m**), keep new bound if accuracy exceeds criterion increment
9. Repeat from step 7 until there are no more bins splits that improve accuracy
10. Measure average performance of final bounds for different random-number seeds
11. Save adaptive histogram equalization (**'AHEO'**) bounds
12. Can also test and save standard histogram equalization (**'HEO'**) bounds

Texture discrimination: train DVs and bounds for spot features

tex_disc_h.m

1. Load cdfs, lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)
2. For task-dependent spot features, set the cdf parameters
3. Load 60 Brodatz and 60 Fabric images; convert to double; for Fabric linearize (**adobe_expand.m**)
4. Apply approximate optics of the human eye (**aply_otf.m**)
5. Convert to LMS cone space (**rgb2lms.m**), down sample for given eccentricity/[lev = 1,2,4,8] (**dsmp.m**)
6. For the task-dependent features load the number of bins and bin bounds (**mk_bb.m**)
7. Randomly sample pairs of patches from same and different texture sheets
8. Normalize each patch to have an average mean across the 3 channels of value of 128 (**ptch_norm.m**)
9. Transform to ABR space (**rot.m**), select color channel A (achromatic channel)
10. Compute spot feature responses for each pair of patches (**Rh.m**)
11. Find the QSVM DV and bound given all the spot feature vectors (**classify_normals.m**)
12. Save the DV function and bound (**'dbndhBFO'**)

Texture discrimination: train DVs and bounds for edge features

tex_disc_e.m

1. Load cdfs, lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)
2. For task-dependent edge features, set the cdf parameters
3. Load 60 Brodatz and 60 Fabric images; convert to double; for Fabric linearize (**adobe_expand.m**)
4. Apply approximate optics of the human eye (**aply_otf.m**)
5. Convert to LMS cone space (**rgb2lms.m**), down sample for given eccentricity/[lev = 1,2,4,8] (**dsmp.m**)
6. For the task-dependent features load the number of bins and bin bounds (**mk_bb.m**)
7. Randomly sample pairs of patches from same and different texture sheets
8. Normalize each patch to have an average mean across the 3 channels of value of 128 (**ptch_norm.m**)
9. Transform to ABR space (**rot.m**), select the A color channel
10. Compute edge feature responses for each pair of patches (**Re.m**)
11. Find the QSVM DV and bound given all the edge feature vectors (**classify_normals.m**)
12. Save the DV function & decision bound (**'dbndeBFO'**)

Texture discrimination: train DVs and bounds for content features

tex_disc_c.m

1. Load cdfs, lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)
2. For task-dependent features, set the cdf parameters
3. Load 60 Brodatz and 60 Fabric images; convert to double; for Fabric linearize (**adobe_expand.m**)
4. Apply approximate optics of the human eye (**aply_otf.m**)
5. Convert to LMS cone space (**rgb2lms.m**), down sample for given eccentricity/[lev = 1,2,4,8] (**dsmp.m**)
6. For the task-dependent features load the number of bins and bin bounds (**mk_bb.m**)
7. Load spot and edge DV coefficients (**'dbndhBFO'**, **'dbndeBFO'**) and functions (**quad2fun.m**)
8. Randomly sample pairs of patches from same and different texture sheets
9. Normalize each patch to have an average mean across the 3 channels of value of 128 (**ptch_norm.m**)
10. Transform to ABR space (**rot.m**), select the A color channel
11. Compute feature responses for each pair of patches (**Rp.m** = DVp, **Rh.m**, **Re.m**)
12. Apply decision variable function to spot and edge responses to obtain DVh & DVe (**dvhfun.m**, **dvefun.m**)
13. Find QSVM DV and bound for [DVp,DVh,Dve] (**classify_normals.m**), save the DV & bound (**'dbndcBFO'**)

Texture discrimination: train DVs and bounds for border features

tex_disc_b.m

1. Load cdfs, lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)
2. Load 60 Brodatz and 60 Fabric images; convert to double; for Fabric linearize (**adobe_expand.m**)
3. Apply approximate optics of the human eye (**aply_otf.m**)
4. Convert to LMS cone space (**rgb2lms.m**), down sample for given eccentricity/[lev = 1,2,4,8] (**dsmp.m**)
5. Randomly sample pairs of patches from the same and different texture sheets: for half of the pairs the second patch is to the right of the reference patch, for half the patch is below the reference patch
6. Compute two border features for each pair: ave energy at border, ave energy away from border (**Rb.m**)
7. Find QSVM DV and bound for (**classify_normals.m**), save the DV & bound (**'dbndbBFO'**)

Texture discrimination: train DVs and bounds for border and content features

tex_disc_bc.m

1. Load cdfs, lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)
2. For task-dependent features, set the cdf parameters
3. Load 60 Brodatz and 60 Fabric images; convert to double; for Fabric linearize (**adobe_expand.m**)
4. Apply approximate optics of the human eye (**aply_otf.m**)
5. Convert to LMS cone space (**rgb2lms.m**), down sample for given eccentricity/[lev = 1,2,4,8] (**dsmp.m**)
6. For the task-dependent features load the number of bins and bin bounds (**mk_bb.m**)
7. Load DV coefficients (**'dbndhBFO'**, **'dbndeBFO'**, **'dbndbBFO'**, **'dbndcBFO'**) and functions (**quad2fun.m**)
8. Randomly sample pairs of patches from same and different texture sheets
9. Normalize each patch to have an average mean across the 3 channels of value of 128 (**ptch_norm.m**)
10. Transform to ABR space (**rot.m**), select the A color channel
11. Compute feature responses for each pair of patches (**Rp.m**, **Rh.m**, **Re.m**, **Rb.m**)
12. Apply DV function to spot and edge responses to obtain DVh & DVe (**dvhfun.m**, **dvefun.m**), DVp = Rp.m
13. Apply decision variable function to border responses to obtain DVb (**dvbfun.m**)
14. Apply decision variable function to [DVp,DVh,DVe] to obtain DVc (**dvcfun.m**)
15. Find QVSM DV for DVb and DVc values (**classify_normals.m**), save the DV & bound (**'dbndbcBFO'**)

Training Task Dependent Features with Natural Images

Texture discrimination: get natural image patches for training (proximity proxy)

nat_near_far_patches.m

1. Load cdfs, lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)
2. Load natural images; convert to double
3. Apply approximate optics of the human eye (**aply_otf.m**)
4. Convert to LMS cone space (**rgb2lms.m**), down sample for given eccentricity/[lev = 1,2,4,8] (**dsmp.m**)
5. Random sample reference patches
6. For each reference patch form two near pairs and two far pairs
7. Do this for all image sets, e.g., Set9_16_num.png num = 1:nimg9
8. Save patch pairs for each image set and desired eccentricity/lev (e.g., **'patch_pairs_91lev.mat'**)

Texture discrimination: learn bins for task-dependent features from natural images

opt_bins_nat.m

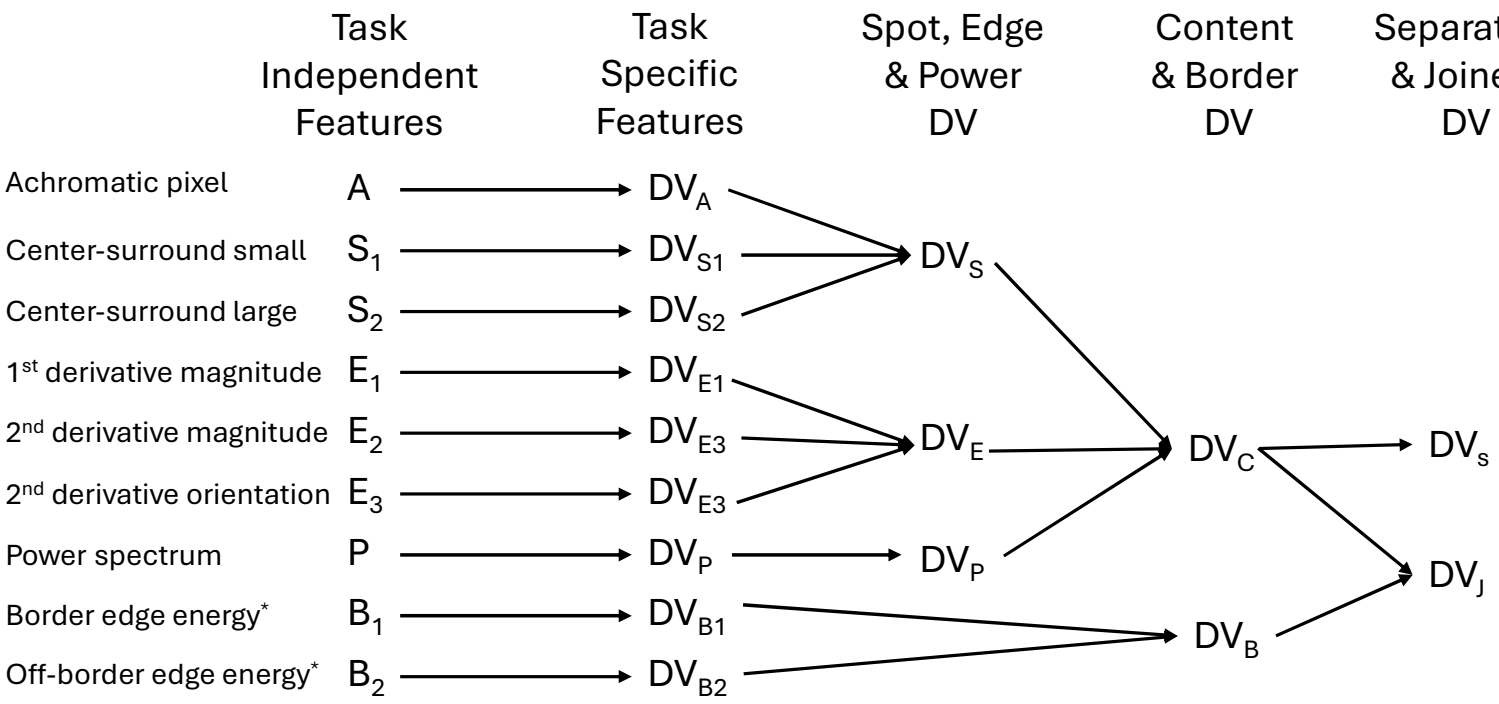
1. Load cdfs and lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)
2. Load near-far natural image patches
3. Use the cdf parameters to get the initial bin bounds; e.g., 2 bins = 3 bounds (**mk_bins.m**)
4. Find the next bin bound to test (**find_bnd.m**)
5. Test the bin bounds including the new bound (**test_bnds_nat.m**), keep if accuracy exceeds criterion increment
6. Repeat from step 4 until there are no more bins splits that improve accuracy
7. Save adaptive histogram equalization (**'AHEO'**) bounds (num1 = btype, num2 = dim, num3 = lev)
8. Can also test and save standard histogram equalization (**'HEO'**) bounds

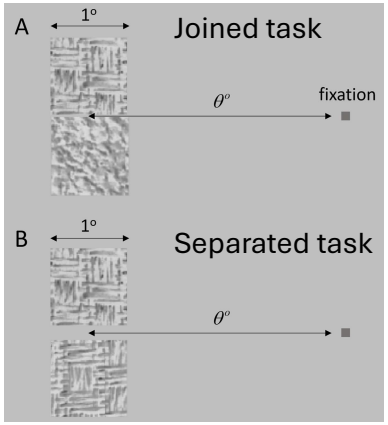
Texture discrimination: train near-far decision variables and bounds 2 (proximity proxy)

nat_near_far_dv_2.m

1. Load cdfs, lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)
2. Set level (lev), set bin type (b)
3. Load patch pairs for each image dataset (e.g., **'patch_pairs_91lev.mat'**) and combine into near and far arrays
4. Normalize each patch to have an average mean across the 3 channels of value of 128 (**ptch_norm.m**)
5. Transform to ABR space (**rot.m**), same abr values for color cdfs, use A channel only
6. Load bin bounds for spot and edge features (**mk_bb.m**)
7. Compute, power and spot DVs (**Rp.m, Rh.m**), Rp.m contrast normalizes and Rh.m does for center-surround rfs
8. Contrast normalization (**cntrst_norm.m**) and compute, edge DVs (**Re.m**)
9. Train spot decision variable (**classify_normals.m**), save dv coefficients (**'dbndhNOlev.mat'**)
10. Train edge decision variable (**classify_normals.m**), save dv coefficients (**'dbndeNOlev.mat'**)
11. Train content decision variable (**classify_normals.m**), save dv coefficients (**'dbndcNOlev.mat'**)
12. Train border decision variable (**classify_normals.m**), save dv coefficients (**'dbndbNOlev.mat'**)
13. Train border & content decision variable (**classify_normals.m**), save dv coefficients (**'dbndbcNOlev.mat'**)

Hierarchical Bayesian Observer (HBO) Model of Texture Discrimination 1





*uses 1st derivative filter

Texture discrimination: single-image self-supervised bc DV & bound (proximity proxy)

self_sup_train_bnd_bc.m

1. Load cdfs, lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)
2. Set level (lev), set image type (itype), set bin type (b)
3. Load images; convert to double; for compressed images linearize (**adobe_expand.m**)
4. Apply approximate optics of the human eye (**aply_otf.m**)
5. Convert to LMS cone space (**rgb2lms.m**), down sample for given eccentricity/[lev = 1,2,4,8] (**dsmp.m**)
6. For the task-dependent features load the number of bins and bin bounds (**mk_bb.m**)
7. Load DV coefficients (**'dbndhNO'**, **'dbndeNO'**, **'dbndbNO'**, **'dbndcNO'**) and functions (**quad2fun.m**)
8. Generate random texture numbers, masks and maps for all trials in a session (**mk_texs.m**, **mk_masks.m**)
9. For each trial, make the GTR image
10. Compute all content similarities and mutual similarities (rho values) for the image (**mk_phi.m**)
11. Extract and process neighboring patches, normalize (**ptch_norm.m**) and transform to abr space (**rot.m**)
12. Compute power (**Rp.m**), spot (**Rh.m**), **dvhfun.m**) and edge (**Re.m**, **dvefun.m**) decision variables (DVs)
13. Compute border (**Rb.m**, **dvbfun.m**) and content (**dvcfun.m**) DVs
14. Compute optimal DV and bound for border and content DV for current trial (**classify_normals.m**, **quad2fun.m**)

15. Vary mutual similarity weight (wm) and criterion (gc) parameters; note that when j = 1 and k = 1 wm = gc = 0 and hence no mutual similarity cue (gc should be set to zero when wm is zero)

16. For each set of parameter values compute near-far and same-different accuracy for all near and far pairs from image (**dvbcfun.m**)

17. Go to step 9

18. Save the average accuracies across patch pairs for each set of parameter values (e.g., 40*162 = 6,480 trials)

Texture discrimination: single-image self-supervised bc-shift DV & bound (proximity proxy)

self_sup_train_bnd_bc_shft.m

1. Load cdfs, lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)
2. Set level (lev), set image type (itype), set bin type (b)
3. Load image images; convert to double; for compressed images linearize (**adobe_expand.m**)
4. Apply approximate optics of the human eye (**aply_otf.m**)
5. Convert to LMS cone space (**rgb2lms.m**), down sample for given eccentricity/[lev = 1,2,4,8] (**dsmp.m**)
6. For the task-dependent features load the number of bins and bin bounds (**mk_bb.m**)
7. Load DV coefficients (**'dbndhNO'**, **'dbndeNO'**, **'dbndbNO'**, **'dbndcNO' , 'dbndbcNO'**) and functions (**quad2fun.m**)
8. Generate random texture numbers, masks and maps for all trials in a session (**mk_texs.m**, **mk_masks.m**)
9. For each trial, make the GTR image
10. Compute all content similarities and mutual similarities (rho values) for the image (**mk_phi.m**)
11. Extract and process neighboring patches, normalize (**ptch_norm.m**) and transform to abr space (**rot.m**)
12. Compute power (**Rp.m**), spot (**Rh.m**), **dvhfun.m**) and edge (**Re.m**, **dvefun.m**) decision variables (DVs)
13. Computer border (**Rb.m**, **dvbfun.m**) and content (**dvcfun.m**) DVs

14. Find opt criterion (copt) for border & content decision variable accuracy

15. Vary mutual similarity weight (wm) and criterion (gc) parameters; note that when j = 1 and k = 1 wm = gc = 0 and hence no mutual similarity cue (gc should be set to zero when wm is zero)

16. For each set of parameter values compute near-far and same-different accuracy for all near and far pairs from image with offset of copt for border & content DV (**dvbcfun.m**)

17. Go to step 9

18. Save the average accuracies across patch pairs for each set of parameter values (e.g., 40*162 = 6,480 trials)

19. Set shftflg = 0 to not use copt

Texture discrimination: single-image self-supervised c-shift DV & bound (proximity proxy)

self_sup_train_bnd_c_shft.m

1. Load cdfs, lms to abr coefficients (**'cdfs_abr_mo13_mo23_cs33_otf.mat'**)
2. Set level (lev), set image type (itype), set bin type (b)
3. Load image images; convert to double; for compressed images linearize (**adobe_expand.m**)
4. Apply approximate optics of the human eye (**aply_otf.m**)
5. Convert to LMS cone space (**rgb2lms.m**), down sample for given eccentricity/[lev = 1,2,4,8] (**dsmp.m**)
6. For the task-dependent features load the number of bins and bin bounds (**mk_bb.m**)
7. Load DV coefficients (**'dbndhNO'**, **'dbndeNO'**, **'dbndbNO'**, **'dbndcNO' , 'dbndbcNO'**) and functions (**quad2fun.m**)
8. Generate random texture numbers, masks and maps for all trials in a session (**mk_texs.m**, **mk_masks.m**)
9. For each trial, make the GTR image
10. Compute all content similarities and mutual similarities (rho values) for the image (**mk_phi.m**)
11. Extract and process neighboring patches, normalize (**ptch_norm.m**) and transform to abr space (**rot.m**)
12. Compute power (**Rp.m**), spot (**Rh.m**), **dvhfun.m**) and edge (**Re.m**, **dvefun.m**) decision variables (DVs)
13. Computer border (**Rb.m**, **dvbfun.m**) and content (**dvcfun.m**) DVs

15. Find opt criterion (copt) for content decision variable accuracy

15. Vary mutual similarity weight (wm) and criterion (gc) parameters, note that when j = 1 and k = 1 wm = gc = 0 and hence no mutual similarity cue (gc should be set to zero when wm is zero)

16. For each set of parameter values compute near-far and same-different accuracy for all near and far pairs from image with offset of copt for content DV (**dvbcfun.m**)

17. Go to step 9

18. Save the average accuracies across patch pairs for each set of parameter values (e.g., 40*162 = 6,480 trials)

19. Set bordflg = 0 to not use border cues

20. Set shftflg = 0 to not use copt